Native handles and file streams

Document #:	P1759R6
Date:	2023-05-15
Project:	Programming Language C++
Audience:	Library

Reply-to:	Elias Kosunen <isocpp@eliaskosunen.com>

Contents

§ 1 Abstract

This paper proposes adding a new typedef to standard file streams: `native_handle_type`. This type is an alias to whatever type the platform uses for its file descriptors: `int` on POSIX, `HANDLE` (`void*`) on Windows, and something else on other platforms. This type is a non-owning handle and has generally sane semantics: default constructability and trivial copyability.

Alongside this, this paper proposes adding a concrete member function: `.native_handle()`, returning a `native_handle_type`, to the following class templates:

— `basic_filebuf`

— `basic_ifstream`

— `basic_ofstream`

— `basic_fstream`

§ 2 Revision History

§ 2.1 R6

— Update wording per LWG feedback

  — Make `native_handle() const` and `noexcept`

  — Add explicit `#define` for feature test macro, and specify, that it should also appear in `<fstream>`

  — Do not require `native_handle_type` to be standard-layout

  — *Returns -> Effects*

  — Other minor changes

§ 2.2 R5

— Update design discussion

  — Add `std::stacktrace_handle` as a type that has a `.native_handle()`, and comparisons to it

— Update wording:

  — Update wording to reference [N4928]

  — *Expects -> Preconditions*

  — Other minor changes based on LEWG feedback

§ 2.3 R4

— Update wording:

  — `native_handle_type` constexpr default constructor -> default constructor

  — Add wording as to how the handle returned by `.native_handle()` is valid only when the file is open

— Update wording to reference the latest standard draft [N4892]

§ 2.4 R3

— Add `std::condition_variable` and [P2146R2] to list of standard types having a `.native_handle()` member function

— Update wording to reference the latest standard draft [N4849], and update references to other P-numbered papers

— Change paper title

§ 2.5 R2

- Minor touches to wording
  - Refine requirements on `native_handle_type` (remove `equality_comparable`, add constexpr default constructability)
  - Fix some broken references using section numbers in the WD
  - Update reference to the WD
- Editorial fixes

2.6 R1

- Make `native_handle_type` be standard layout
- Add precondition (`is_open() == true`) to `.native_handle()`
- Add feature test macro `__cpp_lib_fstream_native_handle`
- Fix errors with opening the file with POSIX APIs in Motivation (see, we need this paper, fstreams are easier to open correctly!)
- Add additional motivating use case in vectored/scatter-gather IO
- `Regular` -> `regular`

Incorporate LEWGI feedback from Cologne (July 2019):

- Move to a member function and member typedef
- Make `native_handle` return value not be mandated to be unique
- Add note about how the presence of the members is required, and not implementation-defined (like for thread)

2.7 R0

Initial revision.

3 Motivation

For some operations, using OS/platform-specific file APIs is necessary. If a user wanted to use these APIs, they're unable to use iostreams without reopening the file.

For example, if one wanted to query the time a file was last modified on POSIX, one would use `fstat`, which takes a file descriptor:

```
int fd = ::open("~/foo.txt", O_RDONLY);
::stat s{};
int err = ::fstat(fd, &s);
std::chrono::sys_seconds                    last_modified                 =
        std::chrono::seconds(s.st_mtime.tv_sec);
```

The Filesystem TS introduced the `status` function returning a `file_status` structure. This doesn't solve our problem, because `std::filesystem::status` takes a path, not a native file descriptor. Using paths is generally discouraged in these sort of situations, because the path may not refer to the same file it referred to previously (the file might've been moved), or the file might not exist anymore at all. In short, using paths is potentially racy.

Also, `std::filesystem::file_status` only contains member functions `type()` and `permissions()`, not one for last time of modification. Extending this structure is out of scope for this proposal, and not feasible for every single possible operation the user may wish to do with OS APIs, of which querying simple file properties is but a small subset.

If the user needs to do a single operation not supported by the standard library, they have to make a choice between using OS APIs exclusively, or reopening the file every time it's necessary. The former is unfortunate from the persective of the standard library and its usefulness. The latter is likely to lead to forgetting to close the file, or running into buffering or synchronization issues, as is the case with C APIs.

```
// Writing the latest modification date to a file
std::chrono::sys_seconds last_modified(int fd) {
    // See above for POSIX implementation using fstat
}

// Today's code

// Option #1:
// Use iostreams by reopening the file
{
    int fd = ::open("~/foo.txt", O_RDONLY); // CreateFile on Windows
    auto lm = last_modified(fd);

    ::close(fd); // CloseFile on Windows
    // Hope the path still points to the file!
    // Need to allocate
    std::ofstream of("~/foo.txt");
    of << std::chrono::format("%c", lm) << '\n';
    // Need to flush
}

// Option #2:
// Abstain from using iostreams altogether
{
    int fd = ::open("~/foo.txt", O_RDWR);
    auto lm = last_modified(fd);

    // Using ::write() is clunky;
    // skipping error handling for brevity
    auto str = std::chrono::format("%c\n", lm);
    ::write(fd, str.data(), str.size());
    // Remember to close!
    // Hope format or push_back doesn't throw
    ::close(fd);
```

```
}

// This proposal
// No need to use platform-specific APIs to open the file
{
    std::ofstream of("~/foo.txt");
    auto lm = last_modified(of.native_handle());
    of << std::chrono::format("%c", lm) << '\n';
    // RAII does ownership handling for us
}
```

The utility of getting a file descriptor (or other native file handle) is not limited to getting the last modification date. Other examples include, but are definitely not limited to:

— file locking (`fcntl()` + `F_SETLK` on POSIX, `LockFile` on Windows)

— getting file status flags (`fcntl()` + `F_GETFL` on POSIX, `GetFileInformationByHandle` on Windows)

— vectored/scatter-gather IO (`vread()`/`vwrite()` on POSIX)

— non-blocking IO (`fcntl()` + `O_NONBLOCK`/`F_SETSIG` on POSIX)

Basically, this paper would make standard file streams interoperable with operating system interfaces, making iostreams more useful in that regard.

An alternative would be adding a lot of this functionality to `fstream` and `filesystem`. The problem is, that some of this behavior is inherently platform-specific. For example, getting the inode of a file is something that only makes sense on POSIX, so cannot be made part of the `fstream` interface, and should only accessible through the native file descriptor.

With [P1031R2] and [P2146R2], we're potentially getting a replacement for iostreams in the standard, or at least facilities complementing them. The author thinks, that even if these papers were to be merged to the standard, the functionality described in this paper would still be useful, as iostreams aren't going anywhere soon.

§ 4 Scope

This paper does *not* propose enabling the construction of a file stream or a file stream buffer from a native file handle. The author is worried of ownership and implementation issues possibly associated with this design.

```
// NOT PROPOSED
#include <fstream>
#include <fcntl.h>

auto fd = ::open(/* ... */);
auto f = std::fstream{fd};
```

This paper also does *not* touch anything related to `FILE*`, namely getting a native handle out of one.

§ 5 Design Discussion

§ 5.1 Implementation-definedness of native handles and presence of them

The wording related to native handles in 33.2.3 [thread.req.native] is as follows:

> Several classes described in this Clause have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined. [*Note:* These members allow implementations to provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — *end note*]

In more plain terms, the presence of native handles in `std::thread`, `std::mutex`, and `std::condition_variable`, is implementation-defined, without a way to query, whether they are provided (except with SFINAE magic). This design is deemed bad by the author and by LEWG, so this paper aims to not do that again.

The wording related to native handles in 19.6.3.3 [stacktrace.entry.obs] is as follows:

> The semantics of this function are implementation-defined.
>
> *Remarks:* Successive invocations of the `native_handle` function for an unchanged `stacktrace_entry` object return identical values.

While this design is a huge step up compared to threads, the author would like to have more strict normative guarantees for this facility.

This paper proposes to:

— set requirements for `native_handle_type`: `semiregularity`, trivial copyability

— define (semi-normatively), what a *native handle* for a file means, and how it behaves

§ 6 Implementation

Implementing this paper should be a relatively trivial task.

Although all implementations surveyed (libstdc++, libc++ and MSVC) use `FILE*` instead of native file descriptors in their `basic_filebuf` implementations, these platforms provide facilites to get a native handle from a `FILE*`; `fileno` on POSIX, and `_fileno` + `_get_osfhandle` on Windows. The following reference implementations use these.

For libstdc++ on Linux:

```
template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
```

```
    // ...
    using native_handle_type = int;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
          // _M_file (__basic_file<char>) has a member function for this
        purpose
        return _M_file.fd();
        // ::fileno(_M_file.file()) could also be used
    }
    // ...
}
```

For libc++ on Linux:

```
template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
    // ...
    using native_handle_type = int;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // __file_ is a FILE*
        return ::fileno(__file_)
    }
    // ...
}
```

For MSVC:

```
template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
    // ...
    using native_handle_type = HANDLE;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // _Myfile is a FILE*
        auto cfile = ::_fileno(_Myfile);
          // _get_osfhandle returns intptr_t, which can be cast to HANDLE
        (void*)
        return static_cast<HANDLE>(::_get_osfhandle(cfile));
    }
    // ...
}
```

For all of these cases, implementing `.native_handle()` for `ifstream`, `ofstream` and `fstream` is trivial:

```
template <class CharT, class Traits>
class basic_ifstream : public basic_istream<CharT, Traits> {
    // ...
    using native_handle_type =
        typename basic_filebuf<CharT, Traits>::native_handle_type;
    // ...
```

```
    native_handle_type native_handle() {
        return rdbuf()->native_handle();
    }
};

// Repeat for ofstream and fstream
```

§ 7 Technical Specifications

The wording is based on [N4928].

Note to editor: In the wording below, replace the character ❓ with the appropriate section or note number.

§ 7.1 Feature test macro

Note to editor: Add the macro below to the appropriate place in 17.3.2 [version.syn], respecting alphabetical order.
Also, set the value of the macro to the date of adoption.

```
#define __cpp_lib_fstream_native_handle 202XXXL  // also in <fstream>
```

§ 7.2 Add the following section into *File-based streams* 31.10 [file.streams]

This section is to come between 31.10.1 [fstream.syn] and 31.10.2 [filebuf].

> **❓.❓.❓ Native handles [file.native]**
>
> Several classes described in [file.streams] have a member `native_handle_type`.
>
> The type `native_handle_type` represents a platform-specific *native handle* to a file. It is trivially copyable and models `semiregular`.
>
> [*Note ❓:* For operating systems based on POSIX, `native_handle_type` is `int`. For Windows-based operating systems, `native_handle_type` is `HANDLE`. — *end note*]

§ 7.3 Modify *Class template* `basic_filebuf` 31.10.2 [filebuf]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class basic_filebuf : public basic_streambuf<charT, traits> {
  public:
    using char_type   = charT;
    using int_type    = typename traits::int_type;
    using pos_type    = typename traits::pos_type;
```

```
    using off_type    = typename traits::off_type;
    using traits_type = traits;
+       using native_handle_type = implementation-defined; // see
        [file.native]

    // ...

    // [filebuf.members], members
    bool is_open() const;
    basic_filebuf* open(const char* s, ios_base::openmode mode);
    basic_filebuf* open(const filesystem::path::value_type* s,
                        ios_base::openmode mode);  // wide systems only;
        see 31.10.1
    basic_filebuf* open(const string& s,
                        ios_base::openmode mode);
    basic_filebuf* open(const filesystem::path& s,
                        ios_base::openmode mode);
    basic_filebuf* close();
+   native_handle_type native_handle() const noexcept;

    // ...
  }
}
```

§ 7.4 Modify *Class template* `basic_filebuf` 31.10.2 [filebuf]

An instance of `basic_filebuf` behaves as described in [filebuf] provided `traits::pos_type` is `fpos<traits::state_type>`. Otherwise the behavior is undefined.

The file associated with a `basic_filebuf` has an associated value of type `native_handle_type`, called the native handle of that file. This native handle can be obtained by calling the member function `native_handle`.

For any opened `basic_filebuf f`, the native handle returned by `f.native_handle()` is invalidated when `f.close()` is called, or `f` is destroyed.

In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as `a_codecvt` in the following subclauses, obtained as if by…

§ 7.5 Add to the end of *Member functions* 31.10.2.4 [filebuf.members]

Note: This would come after the defintion of `basic_filebuf::close()`.

```
native_handle_type native_handle() const noexcept;
```

Preconditions: `is_open()` is `true`.

Returns: The native handle associated with `*this`.

§ 7.6 Modify *Class template* `basic_ifstream` 31.10.3 [ifstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class basic_ifstream : public basic_istream<charT, traits> {
  public:
    using char_type   = charT;
    using int_type    = typename traits::int_type;
    using pos_type    = typename traits::pos_type;
    using off_type    = typename traits::off_type;
    using traits_type = traits;
+   using native_handle_type =
+     typename basic_filebuf<charT, traits>::native_handle_type;

    // ...

    // [ifstream.members], members
    basic_filebuf<charT, traits>* rdbuf() const;
+   native_handle_type native_handle() const noexcept;

    bool is_open() const;
    // ...
  }
}
```

§ 7.7 Add to *Member functions* 31.10.3.4 [ifstream.members]

`basic_filebuf<charT, traits>* rdbuf() const;`

Returns: `const_cast<basic_filebuf<charT, traits>*>(addressof(sb))`.

`native_handle_type native_handle() const noexcept;`

Effects: Equivalent to: `return rdbuf()->native_handle();`

`bool is_open() const;`

Returns: `rdbuf()->is_open()`.

§ 7.8 Modify *Class template* `basic_ofstream` 31.10.4 [ofstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class basic_ofstream : public basic_ostream<charT, traits> {
  public:
    using char_type   = charT;
    using int_type    = typename traits::int_type;
```

```
    using pos_type    = typename traits::pos_type;
    using off_type    = typename traits::off_type;
    using traits_type = traits;
+   using native_handle_type =
+     typename basic_filebuf<charT, traits>::native_handle_type;

    // ...

    // [ofstream.members], members
    basic_filebuf<charT, traits>* rdbuf() const;
+   native_handle_type native_handle() const noexcept;

    bool is_open() const;
    // ...
  }
}
```

§ 7.9 Add to *Member functions* 31.10.4.4 [ofstream.members]

Note: These modifications are identical to those done to 31.10.3.4 [ifstream.members] above

`basic_filebuf<charT, traits>* rdbuf() const;`

Returns: `const_cast<basic_filebuf<charT, traits>*>(addressof(sb))`.

`native_handle_type native_handle() const noexcept;`

Effects: Equivalent to: `return rdbuf()->native_handle();`

`bool is_open() const;`

Returns: `rdbuf()->is_open()`.

§ 7.10 Modify *Class template* `basic_fstream` 31.10.5 [fstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class basic_fstream : public basic_iostream<charT, traits> {
  public:
    using char_type   = charT;
    using int_type    = typename traits::int_type;
    using pos_type    = typename traits::pos_type;
    using off_type    = typename traits::off_type;
    using traits_type = traits;
+   using native_handle_type =
+     typename basic_filebuf<charT, traits>::native_handle_type;

    // ...

    // [fstream.members], members
```

```
    basic_filebuf<charT, traits>* rdbuf() const;
+   native_handle_type native_handle() const noexcept;
+
    bool is_open() const;
    // ...
  }
}
```

§ 7.11 Add to *Member functions* 31.10.5.4 [fstream.members]

Note: These modifications are identical to those done to 31.10.3.4 [ifstream.members] and 31.10.4.4 [ofstream.members] above

`basic_filebuf<charT, traits>* rdbuf() const;`

Returns: `const_cast<basic_filebuf<charT, traits>*>(addressof(sb))`.

`native_handle_type native_handle() const noexcept;`

Effects: Equivalent to: `return rdbuf()->native_handle();`

`bool is_open() const;`

Returns: `rdbuf()->is_open()`.

§ 8 Acknowledgements

Thanks to Jonathan Wakely for reviewing the wording for R3 of this paper.

Thanks to Niall Douglas for feedback, encouragement and ambitious suggestions for this paper.

Thanks to the rest of the co-authors of [P1750R1] for the idea after cutting this functionality out, especially to Jeff Garland for providing a heads-up about a possible ABI-break that I totally would've missed, even though it ended up being a non-issue.

Thanks to Michael Park for his paper markup framework [mpark/wg21].

§ 9 References

[mpark/wg21] mpark/wg21 on GitHub.
https://github.com/mpark/wg21

[N4849] Richard Smith. 2020-01-14. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4849

[N4892] Richard Smith. 2022. Working Draft, Standard for Programming Language C++.
https://wg21.link/N4892

[N4928] Thomas Köppe. 2022. Working Draft, Standard for Programming Language C++.
https://wg21.link/N4928
[P1031R2] Niall Douglas. 2019. Low level file i/o library.
https://wg21.link/p1031r2
[P1750R1] Klemens Morgernstern, Jeff Garland, Elias Kosunen, and Fatih Bakir. 2019. A Proposal to Add Process Management to the C++ Standard Library.
https://wg21.link/p1750r1
[P2146R2] Amanda Kornoushenko. 2020. Modern std::byte stream IO for C++.
https://wg21.link/p2146r2